

PROJECT REPORT

STUDENT INFORMATION SYSTEM

Programming in Java

Submitted By:	AYUSH RAJ SINGH
Register Number:	25BAI11478
Program / Branch:	CSE (AI & ML)
Semester:	Fall Semester
Faculty:	Mo. Rizwan
Academic Year:	2026-27

1. Introduction

The Student Information System is a Java-based terminal application developed to manage student records in a simple, structured, and reliable manner. The application provides a menu-driven console interface through which the user can add, display, search, update, and delete student information.

The implementation uses Core Java concepts including classes and objects, encapsulation, ArrayList, methods, loops, conditional statements, exception handling, file handling, and object serialization. Student records are stored locally in `students.dat` so that saved information can be restored when the application is started again.

2. Problem Statement

Manual management of student records can become difficult as the number of records increases. It may result in duplicate IDs, incorrect GPA values, difficulty locating a student, and loss of information. The proposed system provides a terminal-based solution for maintaining student ID, name, department, and GPA with validation and persistent local storage.

3. Functional Requirements

ID	Requirement Description
FR-01	Add a new student record.
FR-02	Prevent duplicate student IDs.
FR-03	Display all registered students.
FR-04	Search a student using ID.
FR-05	Update name, department, and optionally GPA.
FR-06	Delete a student after confirmation.
FR-07	Accept GPA only from 0.0 to 4.0.
FR-08	Handle invalid integer and decimal input.
FR-09	Load saved records from <code>students.dat</code> .
FR-10	Save records using Java serialization when exiting.

4. Non-Functional Requirements

- **Usability:** Clear menu-driven terminal interaction.
- **Reliability:** Invalid input is handled using validation and exception handling.
- **Maintainability:** Operations are separated into modular methods.
- **Performance:** Suitable response time for student datasets.
- **Persistence:** Records remain available after saving and restarting the application.

5. System Architecture

The application follows a console-based architecture consisting of user interaction, application logic, in-memory data storage, and local file storage.

```
[ User Console Menu ] → [ StudentManagementSystem ] → [ ArrayList  
                        ] → [ students.dat ]
```

6. Design Specifications

6.1 System Components & Classes

Class	Attributes / Methods
Student	- id: String - name: String - department: String - gpa: double + getters / setters + toString()
StudentManagementSystem	- FILE_NAME: String - students: List<Student> + main() + addStudent() + viewAllStudents() + searchStudent() + updateStudent() + deleteStudent()

6.2 Key Workflow

```
START → loadData() → Display Main Menu → Read Option (1-6) → Process Action →  
Return to Menu → Option 6: saveData() → END
```

7. Design Decisions & Rationale

Decision	Rationale
Terminal Interface	Focuses core project scope on Java OOP fundamentals without GUI complexity.
ArrayList Data Structure	Provides dynamic, lightweight in-memory record manipulation.
Java Object Serialization	Enables seamless file persistence to students.dat without SQL databases.

8. Testing Approach & Validation

Testing was performed across functional boundaries including duplicate prevention, edge-case GPA values, and persistence verification.

TC ID	Test Input/Action	Expected Result
TC-01	Valid new student entry	Record added successfully.
TC-02	Duplicate student ID entered	Duplicate ID rejected with warning.
TC-03	GPA entered outside 0.0–4.0 range	Input re-promoted with validation error.
TC-04	Display all students action	All active records formatted in terminal table.
TC-05	Search by existing ID	Matching record retrieved and displayed.
TC-06	Delete student with 'y' confirmation	Record purged from list and storage.
TC-07	Restart application after Save & Exit	Saved records retrieved from students.dat.

9. Key Takeaways & Future Scope

- Applied object-oriented principles (Encapsulation, Data Hiding, Serialization).
- Implemented robust validation for terminal menu interaction.
- Future Scope:** Database integration (MySQL/PostgreSQL), GUI interface using JavaFX, and role-based login capabilities.

10. Conclusion

The Student Information System demonstrates how Core Java can be used to build a practical terminal-based record management application. The system fulfills all requirements including CRUD operations, input validation, ID lookup, and file persistence.